Project 2 CO-2 Report – Lab2-1 & Lab2-2

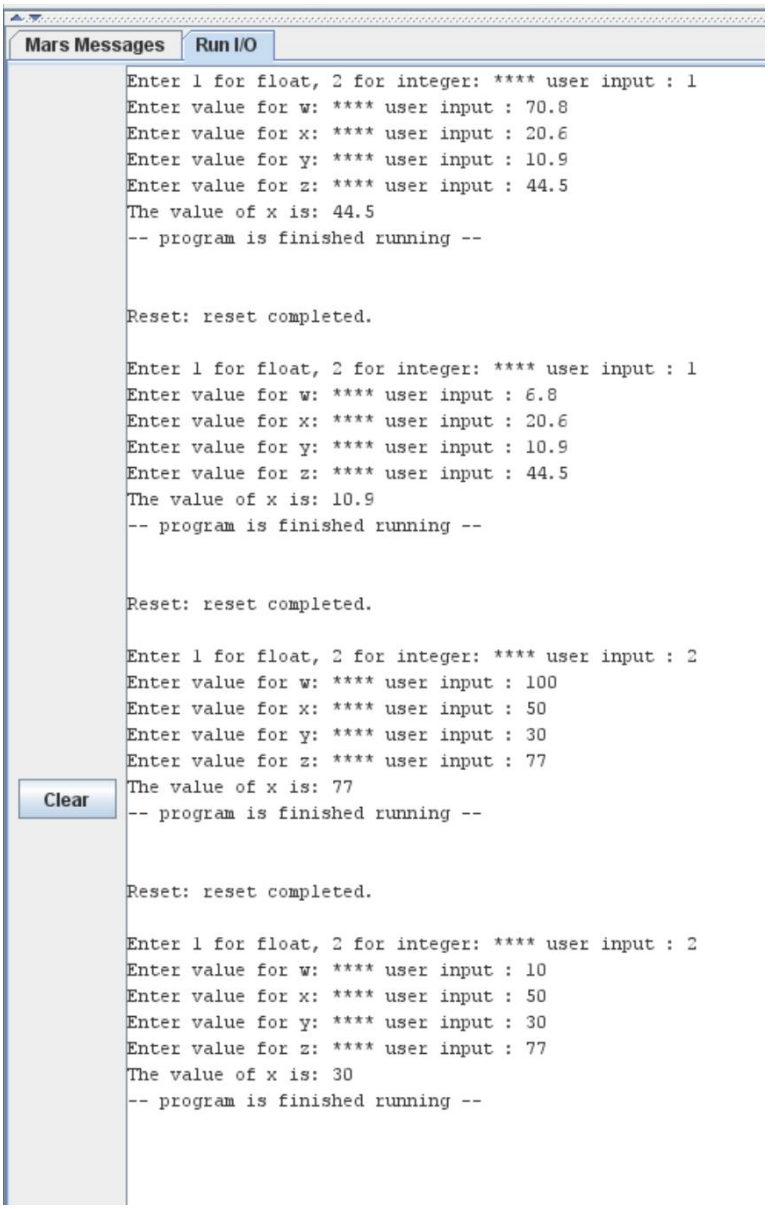
1. Austin Sternberg
2. Code Files:
    1. lab2-1.asm
    2. lab2-2.asm
3. Program Implementation:

    Lab2-1: To solve the problem asked, I started by building an initial prompt for what type of data the user wishes to enter, covered from lines 41-56. This is done by simply asking for a 1 or 2 which corresponds to either float or integer numbers. Lines 57-66 work for any other number, kicking the user back to the original prompt. Given user input, the code either jumps to lines 68-125 or lines 129-180. Lines 68-125 cover the input, storing and processing of float variables. For each number, the user is prompted individually, then the number is stored in a pre-determined memory location, then is restored to registers as needed for the calculation and evaluation that takes place between lines 105-118. If (x-y) is greater than or equal to w the value of x is set to y. If (x-y) is less than w, we jump to lines 120-125, and store the value of z in the memory of x. From either of these sections, we jump to the end_label section of lines 195-205, which handle printing out the current value of the memory space holding x. From there, we exit the program. Integer processing works similarly, though all the values remain in the registers during our computations and processing. Lines 130-165 handle taking integer values for variables w, x, y, and z, then we calculate (x-y) and compare to w in lines 166-177. If (x-y) is greater or equal to w, we set x to the value of y, then jump to output. If (x-y) is less than w, we jump to to lines 178-

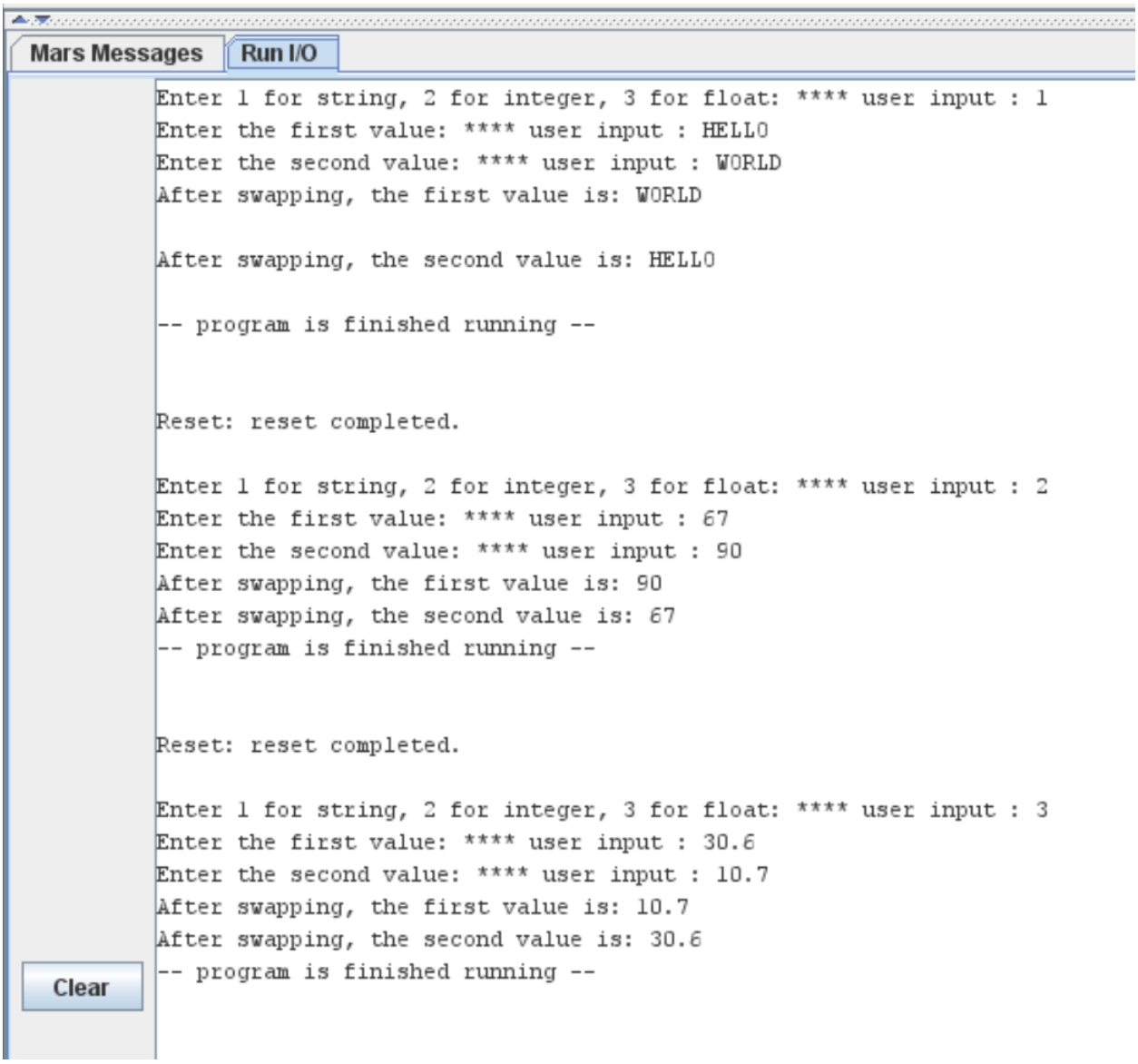
181, where the value of x is set to the value of z. From there lines 184-193 involve printing the integer value processed to output, then we jump to the end to exit the program.

Lab2-2: Similarly to the first part, I am using another prompt of the user for a value of either 1, 2, or 3 to determine input type. Lines 40-58 work to support this selection process, mapping a 1 to string input, a 2 to integer input, and a 3 to float input. Lines 59-66 subsequently work to address any other values entered, kicking the user back to the original prompt. For strings, my input processing and printing takes place between lines 69-153, where I start by reading in two string values from the user individually. From there the code in lines 90-128 works to copy the contents of string 1 to the temporary string, then copy the contents of string 2 to string 1, and finally copy the contents of the temporary string to string 2. This effectively swaps our two string inputs, and we move to lines 129-152 to print both string 1 and string 2 to the console. Input swapping is handled between lines 155-202, where we prompt for two integers between lines 157-172. The registers are then swapped across lines 175-177 using a third temporary register, in that I store the first integer to the temp, store the second integer to the first, then store the temp to the second integer. From there lines 179-202 work to print the values of the first and second integers to console. Float processing works basically identically, with lines 206-223 prompting and storing the two float values. Then lines 225-233 swap the two using another temporary float storage location, and then lines 235-259 print those current floating-point values to console. In each case, the code jumps to the ending code to exit after printing to console.

4. Lab2-1:


Mars Messages
Run I/O
Clear
Enter 1 for float, 2 for integer: **** user input : 1
Enter value for w: **** user input : 70.8
Enter value for x: **** user input : 20.6
Enter value for y: **** user input : 10.9
Enter value for z: **** user input : 44.5
The value of x is: 44.5
-- program is finished running --
Reset: reset completed.
Enter 1 for float, 2 for integer: **** user input : 1
Enter value for w: **** user input : 6.8
Enter value for x: **** user input : 20.6
Enter value for y: **** user input : 10.9
Enter value for z: **** user input : 44.5
The value of x is: 10.9
-- program is finished running --
Reset: reset completed.
Enter 1 for float, 2 for integer: **** user input : 2
Enter value for w: **** user input : 100
Enter value for x: **** user input : 50
Enter value for y: **** user input : 30
Enter value for z: **** user input : 77
The value of x is: 77
-- program is finished running --
Reset: reset completed.
Enter 1 for float, 2 for integer: **** user input : 2
Enter value for w: **** user input : 10
Enter value for x: **** user input : 50
Enter value for y: **** user input : 30
Enter value for z: **** user input : 77
The value of x is: 30
-- program is finished running --

Lab2-2:



5. This lab was a step up from the last one, especially since it needed to support multiple different inputs. Strings are honestly very similar to integers, expect that you need to use loops to print them and move them, which isn't the most straightforward thing to do in MIPs. The hard part came with the first half, the need to use floating numbers was considerably harder to make work. I was getting huge errors and incorrect output for a while till I figured out how to make it work, and even then, I am not convinced my

solution is the most efficient. Those numbers are considerably more difficult to do computations with compared to normal integers. The other difficult part of this assignment was figuring out whether I should use SPIMs (as the instructions stated) or use MARS (as we had been shown how to use in class). I figured it best to make the code function in both and therefore tested and confirmed that it does both ways.